CHESS IN JS

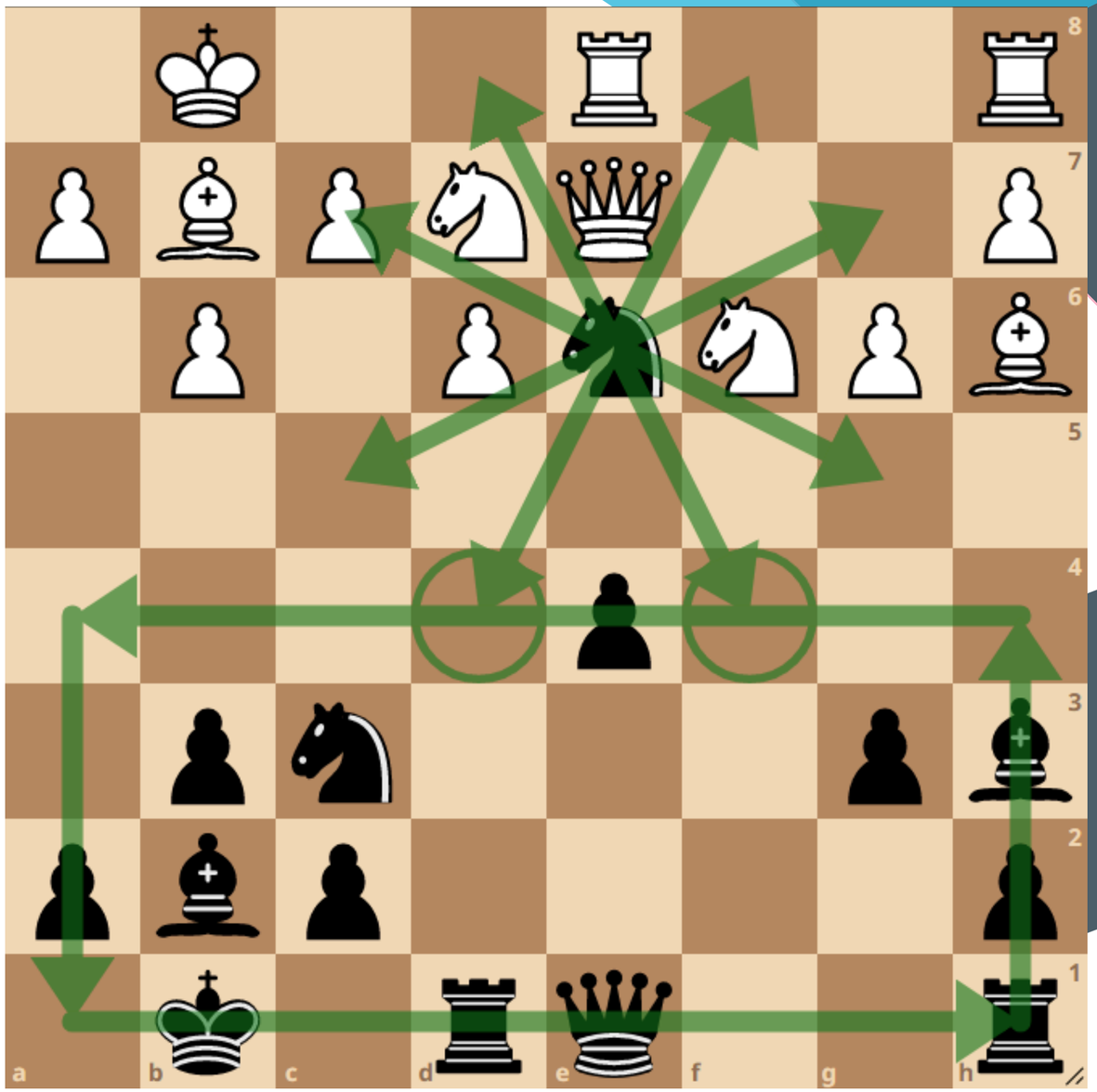
Arnav Devalapally

October 29th 2022

About
INTRODUCTION

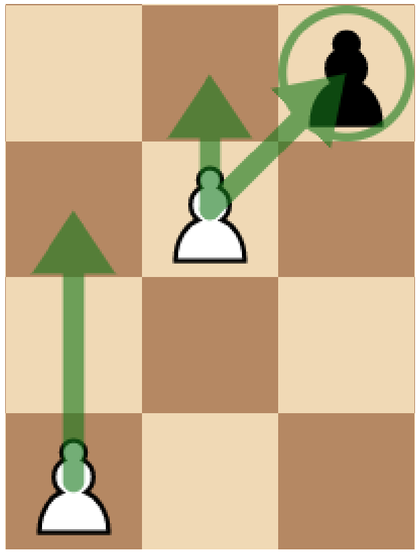
Chess is a two player strategy game played on a 64 x 64 square chessboard, with the goal of capturing the opponent's king. Although this may seem simple, the underlying rules and strategies make chess a very complex game.

Despite being over 1500 years old, chess is still played by millions everyday



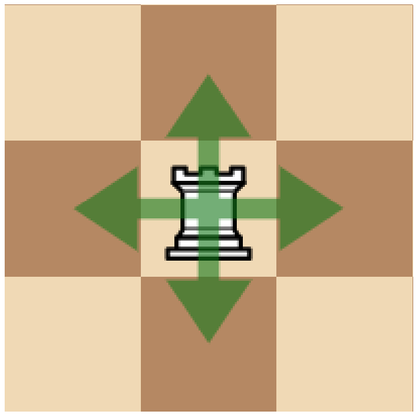
Robert Byrne vs Bobby Fischer US Championship (1963/64)

Basic Moves

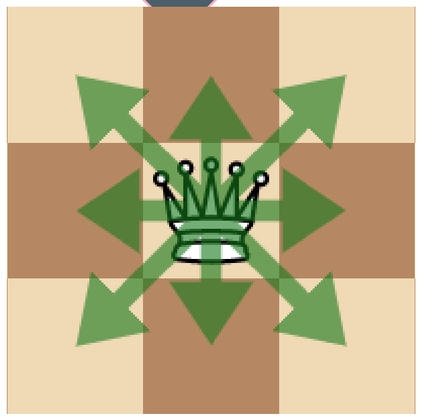


Pawn moves:

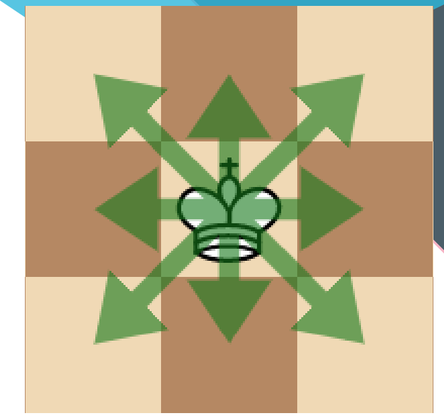
- 1-2 squares on first move
- 1 square on every other move
- captures diagonally



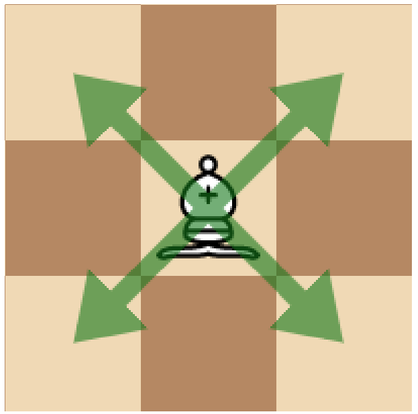
Rook moves horizontally



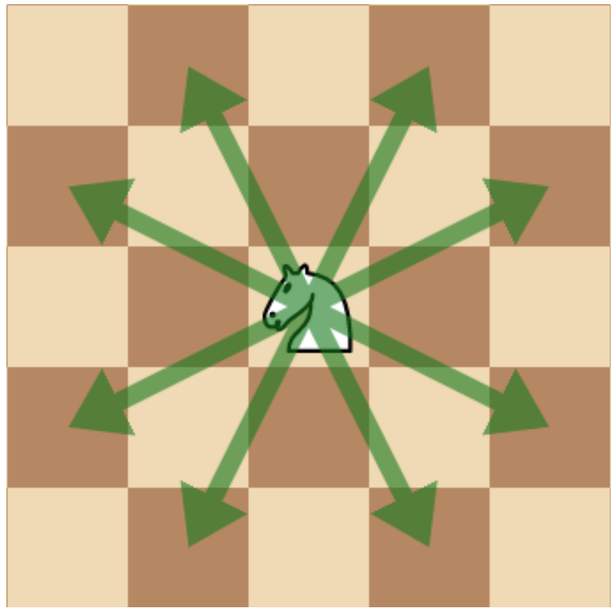
Queen moves horizontally or diagonally



King moves one square horizontally or diagonally



Bishop moves diagonally



Knight moves in an L shape

Designing Chess
MAIN IDEA

TOPIC #01

Basic Plan

TOPIC #02

Hardships

TOPIC #03

Showcase

TOPIC #01

BASIC PLAN

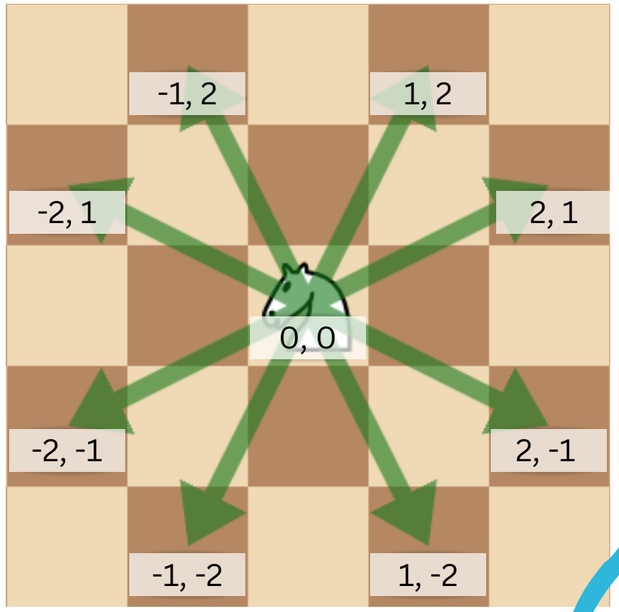
Summarize
BASIC PLAN

Key Point #01
Give an cell id based on the location of each cell. We can then mathematically determine the legality of a move based on the from-cell, to-cell and piece-type.

Key Point #02
Add special moves such as *enpassant,* castle, long castle, pawn promotion

Key Point #03
For every move, check for legality, see if the move checks the king, or if the game is won.

Moving a Knight



Either 2 horizontal and 1 vertical
or
1 horizontal and 2 vertical

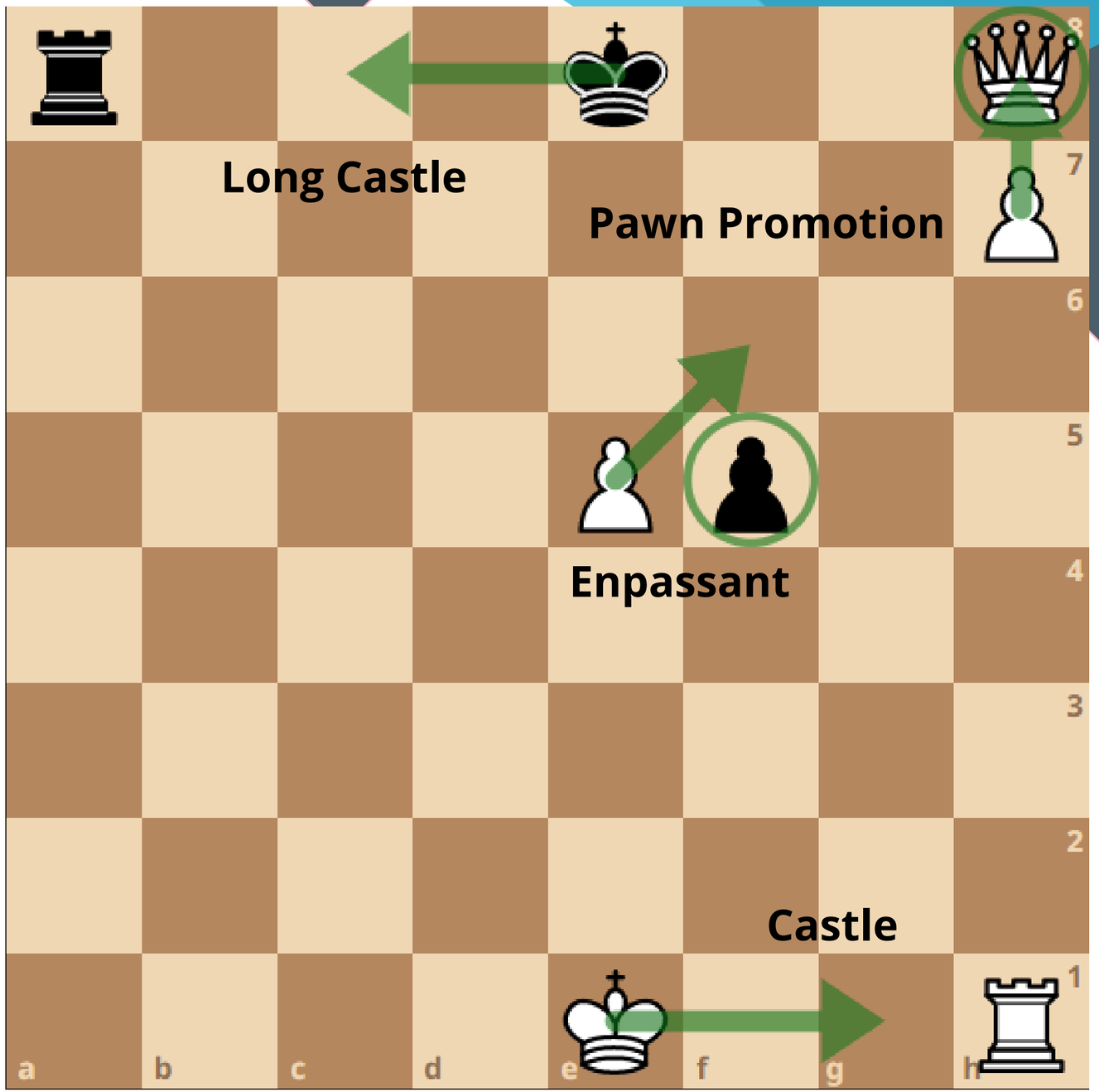
```
isLegalKnightMove = (from, to, color) => {
    const fromRow = parseInt(from.id[1]);
    const fromCol = parseInt(from.id[0]);
    const toRow = parseInt(to.id[1]);
    const toCol = parseInt(to.id[0]);
    if (hasPiece(to) && getPiece(to).className[0] == color) {
        //console.log("Knight move blocked by piece at", to.id);
        return false;
    }
    if (Math.abs(fromRow - toRow) == 2 && Math.abs(fromCol - toCol) == 1) {
        return true;
    }
    if (Math.abs(fromRow - toRow) == 1 && Math.abs(fromCol - toCol) == 2) {
        return true;
    }
    return false;
};
```

Note: In hindsight, a simpler approach would be to check if the Euclidian distance between the points is √5. But this may be difficult to implement due to floating point equality.

Special Moves

Apart from the basic Chess moves that every piece follows, there are also 4 special chess moves that need to be treated differently; namely:

1. Castle
2. Long Castle
3. Enpassant
4. Pawn Promotion



TOPIC #02

HARDSHIPS

Summarize
HARDSHIPS

Key Point #04
Castle cannot be done when a king moves once, this means that for the same board state, a move may or may not be possible. A similar but more challenging problem is also present for enpassant.

Key Point #05
Rook, Queen, Bishop moves require there to be no pieces in the way

Key Point #06
Pins and Discovered checks pose a huge problem when seeing if the king is checked.

Game data & Rook

```
let gameData = {
    turn: 'W',
    gameEnded: false,
    whiteKingMoved: false,
    blackKingMoved: false,
    whiteEnpassant: null,
    blackEnpassant: null,
};
```

Adding variables to store board data

- If King moved, Castle and Long Castle is Illegal
- Enpassant can only be done on the move after a pawn moves twice, so the variable stores the column of the pawn that can be captured through enpassant

```
isLegalRookMove = (from, to, color) => {
    const fromRow = parseInt(from.id[1]);
    const fromCol = parseInt(from.id[0]);
    const toRow = parseInt(to.id[1]);
    const toCol = parseInt(to.id[0]);
    if (hasPiece(to) && getPiece(to).className[0] == color) {
        //console.log("Rook move blocked by piece at", to.id);
        return false;
    }
    if (fromRow == toRow) {
        for (let i = Math.min(fromCol, toCol) + 1; i < Math.max(fromCol, toCol); i++) {
            if (hasPiece(elem(i + '' + fromRow))) {
                //console.log("Rook move blocked by piece at", i + '' + fromRow);
                return false;
            }
        }
        return true;
    }
    if (fromCol == toCol) {
        for (let i = Math.min(fromRow, toRow) + 1; i < Math.max(fromRow, toRow); i++) {
            if (hasPiece(elem(fromCol + '' + i))){
                //console.log("Rook move blocked by piece at", fromCol + ''  + i);
                return false;
            }
        }
        return true;
    }
    return false;
};
```

Checking if a piece blocks move

Pins and Discovered Checks

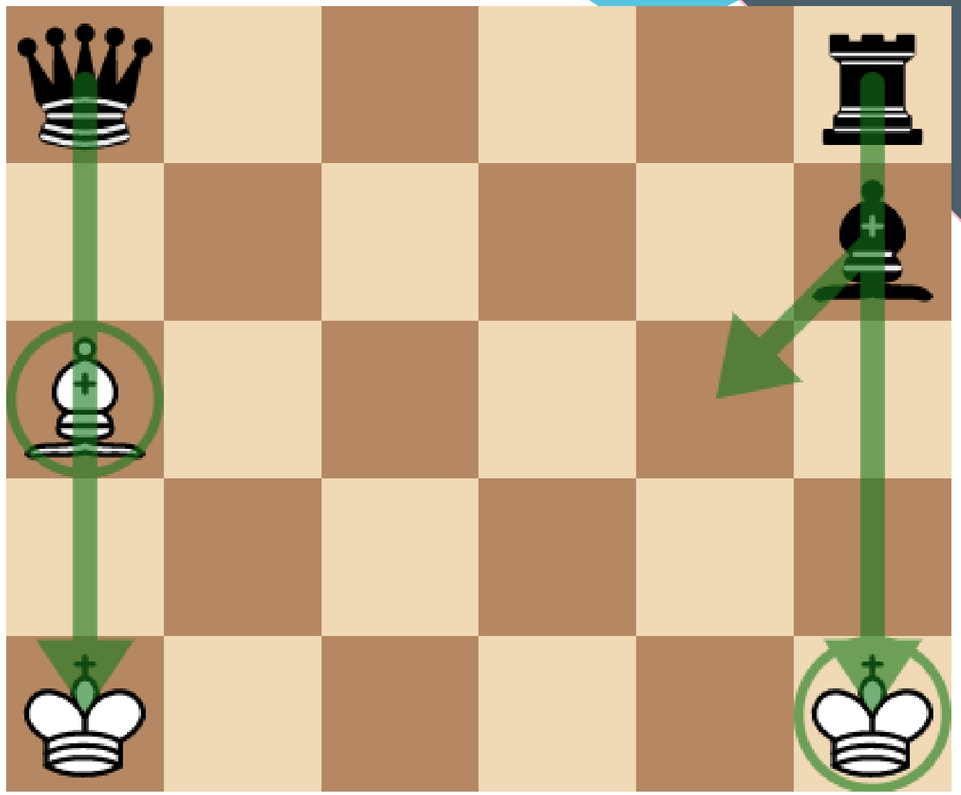
Detecting if a move causes a check seemed to be an easy task at first, I had a few ideas for this:

- Check if the king walks into an attacked square
- Check if the piece after moving checks the king

However, two types of moves namely a Pin and a Discovered Check made this a much more difficult problem.

In the figure, moving the white bishop is illegal since it would cause the black queen to check the king; the king is in check without ever moving it.

Similarly, moving the black bishop causes the rook to check the king; the bishop moves, yet the rook checks the king.



Pin

A pinned piece cannot move

Discovered Check

A piece causes check without moving

Solution to Check - Brute Force

```
isCheck = (color) => {
    let king = document.getElementsByClassName(color + 'K')[0]
    if (king){
        king = king.parentNode;
    } else {
        console.log('Somehow, the king is dead!');
        return false;
    }

    const moves = allLegalMoves();
    for (let i = 0; i < moves.length; i++){
        if (moves[i] == king.id){
            return true;
        }
    }
    return false;
};
```

isCheck function:

Check if there is any legal move that attacks king

```
causesCheck = (from, to, color) => {
    let toPiece = null;
    let fromPiece = getPiece(from);
    from.removeChild(from.children[0]);
    if (hasPiece(to)){
        toPiece = getPiece(to)
        to.removeChild(to.children[0]);
    }
    to.appendChild(fromPiece);

    const check = isCheck(color);

    to.removeChild(to.children[0]);
    from.appendChild(fromPiece);
    if (toPiece != null){
        to.appendChild(toPiece);
    }
    return check;
};
```

1

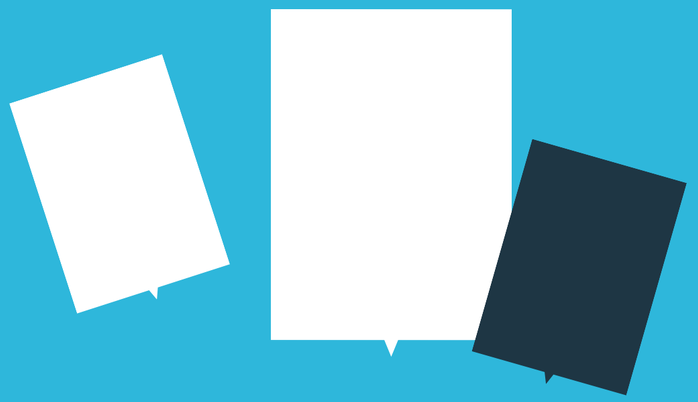
2

3

4

causesCheck function:

1. Move Piece
2. Store Check Status
3. Move Piece Back
4. Return Check Status



DEMO TIME

Developer Notes
FAQ

Key Note 1
Chess.js is a free and public library that allows you to simulate the game with a single command. The code is also more efficient and less prone to bugs than mine.

Key Note 2
Chess is JS is not really a project. At its core, it just does what other websites do, but with a worse GUI.

Key Note 3
The project has a very limited scope of improvement, and isn't innovative at all.

No Libraries used

Chess in JS is written 90% in JavaScript with over 700 lines of dense code. But the same could be achieved with a single import.

The project started on August 9th, yet was only completed in the start of October. It went through 3 versions, and the one showcased was version 2.
While the first two versions were written from scratch, the third one uses the Chess.js library to greatly reduce the size of the code.

The project didn't start as something I wanted to showcase, but rather something I used to learn and explore.

"It is not a Project, but rather a Proof of concept"

Note: Something I want to emphasize, is to always use libraries in a project, it makes the code simpler, improves efficiency and reduces bugs. However, if you have some time, and want to challenge yourself, feel free to do so.

What I Learned

The first version of the code, which ultimately had to be remade due to a problem I discussed previously (*Pins and Discovered Checks)* started before I knew how to code in JavaScript, or even HTML or CSS for that matter.

Now, I consider myself proficient in JS, and that would be mostly thanks to this project. But, more importantly, I learned to code with proper structure and convention, which is important for large projects where readability is very important.

What Next

Although the game is functional, it is not very good looking, or easy to use, and so my next goal is to improve the GUI, or in another sense, use the project to learn CSS.

CLOSING NOTES

This project was loads of fun to code, debug, and run.
I estimate it took about 30 hours to complete, yet was fun all the way, and learned a lot from it.

Check out the source code on my GitHub

https://github.com/D-Arnav/Chess-in-JS

THANK YOU

Arnav Devalapally
2110080023@klh.edu.in